

Detailed Notes

Chapter 4 — Arrays and Strings

Topics Covered

- Single-Dimension Arrays
- Generating a Pointer to an Array
- Passing Single-Dimension Arrays to Functions
 - Strings
 - Common String Library Functions
 - Two-Dimensional Arrays
 - Memory Storage and Size Formula
 - Passing 2D Arrays to Functions
 - Arrays of Strings
 - Multidimensional Arrays
 - Memory Requirements
 - Passing to Functions
 - Indexing Pointers
 - Array Initialization
 - Single-Dimension Initialization
 - String Array Initialization
 - Multidimensional Initialization and Subaggregate Grouping
 - Unsized Array Initializations
- Variable-Length Arrays (C99)
- Quick Reference Summary

Red: First

Yellow: Second

Green: Third

CHAPTER 4 — ARRAYS AND STRINGS

An array is a collection of variables of the same type that are referred to through a common name.

A specific element in an array is accessed by an index.

In C, all arrays consist of contiguous memory locations.

The lowest address corresponds to the first element and the highest address to the last element.

Arrays can have from one to several dimensions.

The most common array is the string, which is simply an array of characters terminated by a null.

Arrays and pointers are closely related — a discussion of one usually refers to the other.

Key Concept	Explanation
Array	A collection of variables of the same type referred to through a common name
Index	A value placed in square brackets used to access a specific element of an array; always starts at 0 in C
Contiguous memory	All array elements are stored in adjacent memory locations
String	The most common 1D array — a null-terminated character array

SINGLE-DIMENSION ARRAYS

The general form for declaring a single-dimension array is:

```
type var_name[size];
```

Part	Meaning
type	The base type of the array — the type of each element stored in the array
var_name	The name of the array variable
size	The number of elements the array will hold

Like other variables, arrays must be explicitly declared so that the compiler can allocate space for them in memory.

Example — declaring and accessing a 100-element double array:

```
double balance[100]; /* declares a 100-element array of type double */  
  
balance[3] = 12.23; /* assigns 12.23 to element at index 3 */
```

- In C, all arrays have 0 as the index of their first element
- `char p[10]` declares a character array with 10 elements: `p[0]` through `p[9]`
- In C89, the size of an array must be a constant expression — the size is fixed at compile time
- C99 allows arrays whose sizes are determined at run time (variable-length arrays — described later)

Example — loading an integer array with values 0 through 99:

```
#include <stdio.h>
int main(void)
{
    int x[100]; /* declares a 100-integer array */
    int t;
    /* load x with values 0 through 99 */
    for(t=0; t<100; ++t) x[t] = t;
    /* display contents of x */
    for(t=0; t<100; ++t) printf("%d ", x[t]);
    return 0;
}
```

Memory Storage of Single-Dimension Arrays

The amount of storage required to hold an array is directly related to its type and size. For a single-dimension array, the total size in bytes is computed as:

$$\text{total bytes} = \text{sizeof}(\text{base type}) \times \text{length of array}$$

Single-dimension arrays are essentially lists stored in contiguous memory locations in index order. For example, a char array of 7 elements starting at memory location 1000:

Element	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]
Address	1000	1001	1002	1003	1004	1005	1006

NOTE *C has NO bounds checking on arrays. You could overwrite either end of an array and write into some other variable's data or even into the program's code. As the programmer, it is your job to provide bounds checking where needed.*

Example of dangerous unchecked code that compiles without error but causes array overrun:

```
int count[10], i;
/* this causes count to be overrun — WRONG! */
for(i=0; i<100; i++) count[i] = i;
```

```
/* count is only 10 elements; accessing i=10..99 corrupts memory */
```

GENERATING A POINTER TO AN ARRAY

You can generate a pointer to the first element of an array by simply specifying the array name without any index.

```
int sample[10];
int *p;
p = sample; /* p now holds the address of sample[0] */
```

Expression	Meaning	Notes
sample	Pointer to first element of the array	Used in professionally written C code
&sample[0]	Address of the first element	Same result as using sample alone, but almost never seen in practice

- Both sample and &sample[0] produce the same result — the address of the first element
- In professionally written C code, you will almost never see &sample[0]
- The array name used without an index always evaluates to the address of the first element

Complete example — assigning array address to a pointer:

```
int *p;
int sample[10];
p = sample; /* p points to sample[0] */
/* p and &sample[0] are equivalent — both give the same address */
```

PASSING SINGLE-DIMENSION ARRAYS TO FUNCTIONS

In C, you cannot pass an entire array as an argument to a function. You can, however, pass a pointer to an array by specifying the array's name without an index. For example, the following program fragment passes the address of array i to func1():

```
int main(void)
{
    int i[10];
    func1(i); /* passes pointer to first element — array name without index */
    /* ... */
}
```

If a function receives a pointer to a single-dimension array, you can declare its formal parameter in one of THREE ways:

```
/* Method 1: as a pointer */
void func1(int *x)
{
    /* ... */
}

/* Method 2: as a sized array */
void func1(int x[10])
{
    /* ... */
}

/* Method 3: as an unsized array */
void func1(int x[])
{
    /* ... */
}
```

Method	Declaration	How Compiler Treats It
1 — Pointer	int *x	Most explicit; the pointer itself is declared directly
2 — Sized array	int x[10]	Standard array declaration; compiler generates code to receive a pointer, NOT a 10-element array
3 — Unsized array	int x[]	Modified array declaration; specifies an int array of unspecified length

- All three declaration methods produce similar results — each tells the compiler that an integer pointer is going to be received
- The length of the array does not matter as far as the function is concerned because C performs NO bounds checking
- Even `void func1(int x[32])` works — the compiler generates code to receive a pointer, not a 32-element array

NOTE *The size specified in a sized-array parameter declaration is completely ignored by the compiler. It does NOT create an array of that size. It is just another way of saying 'receive a pointer to int'.*

STRINGS

By far the most common use for the one-dimensional array is as a character string. In C, a string is a null-terminated character array. A null is zero (`\0`). Thus, a string contains the characters that make up the string followed by a null. The null-terminated string is the only type of string defined by C.

NOTE *C++ also defines a string class called `string`, which provides an object-oriented approach to string handling, but it is not supported by C.*

When declaring a character array that will hold a string, you need to declare it to be ONE CHARACTER LONGER than the largest string that it will hold. For example, to declare an array `str` that can hold a 10-character string:

```
char str[11]; /* holds up to 10 characters + 1 null terminator */
```

- Specifying 11 for the size makes room for the null at the end of the string
- A string constant is a list of characters enclosed in double quotes: `"hello there"`
- You do not need to add the null to the end of string constants manually — the compiler does this automatically

Common String Library Functions

C supports a wide range of functions that manipulate strings. These functions require the standard header `<string.h>`. The most common are:

Function	Description and Return Value
strcpy(s1, s2)	Copies string <code>s2</code> into <code>s1</code> . Returns <code>s1</code> .
strcat(s1, s2)	Concatenates (appends) <code>s2</code> onto the end of <code>s1</code> . Returns <code>s1</code> .
strlen(s1)	Returns the length of <code>s1</code> as an integer — does NOT count the null terminator.
strcmp(s1, s2)	Compares <code>s1</code> and <code>s2</code> lexicographically. Returns 0 if equal; less than 0 if <code>s1 < s2</code> ; greater than 0 if <code>s1 > s2</code> .
strchr(s1, ch)	Returns a pointer to the first occurrence of character <code>ch</code> in <code>s1</code> . Returns NULL if not found.

Function	Description and Return Value
strstr(s1, s2)	Returns a pointer to the first occurrence of string s2 in s1. Returns NULL if not found.

Example program illustrating all six string functions:

```
#include <stdio.h>
#include <string.h>
int main(void)
{
    char s1[80], s2[80];
    gets(s1); /* read first string */
    gets(s2); /* read second string */
    printf("lengths: %d %d\n", strlen(s1), strlen(s2));
    if(!strcmp(s1, s2)) printf("The strings are equal\n");
    strcat(s1, s2); /* append s2 to end of s1 */
    printf("%s\n", s1);
    strcpy(s1, "This is a test.\n");
    printf(s1);
    if(strchr("hello", 'e')) printf("e is in hello\n");
    if(strstr("hi there", "hi")) printf("found hi");
    return 0;
}
```

If both strings entered are "hello", the output is:

```
lengths: 5 5
The strings are equal
hellohello
This is a test.
e is in hello
found hi
```

NOTE *strcmp()* returns false (0) if the strings are equal. When testing for equality, be sure to use the logical ! operator to reverse the condition: *if(!strcmp(s1, s2)) ...*

TWO-DIMENSIONAL ARRAYS

C supports multidimensional arrays. The simplest form of the multidimensional array is the two-dimensional array. A two-dimensional array is, essentially, an array of one-dimensional arrays.

Declaration — declaring a 2D integer array d of 10 rows and 20 columns:

```
int d[10][20];
/* C uses separate brackets for each dimension — NOT commas like some other languages */
/* Some languages: d[10,20] — C: d[10][20] */
```

Accessing a specific element — access point row 1, column 2:

```
d[1][2] /* accesses element at row 1, column 2 */
```

Example — loading a 3×4 array with numbers 1 through 12 and printing row by row:

```
#include <stdio.h>
int main(void)
{
    int t, i, num[3][4];
    for(t=0; t<3; ++t)
        for(i=0; i<4; ++i)
            num[t][i] = (t*4)+i+1;
    /* now print them out */
    for(t=0; t<3; ++t) {
        for(i=0; i<4; ++i) printf("%3d ", num[t][i]);
        printf("\n");
    }
    return 0;
}
```

Visualization of the num[3][4] array — values 1 through 12:

	Col 0	Col 1	Col 2	Col 3
Row 0 (t=0)	num[0][0]=1	num[0][1]=2	num[0][2]=3	num[0][3]=4
Row 1 (t=1)	num[1][0]=5	num[1][1]=6	num[1][2]=7	num[1][3]=8
Row 2 (t=2)	num[2][0]=9	num[2][1]=10	num[2][2]=11	num[2][3]=12

Memory Storage of Two-Dimensional Arrays

- Two-dimensional arrays are stored in a row-column matrix
- The LEFT index indicates the ROW; the RIGHT index indicates the COLUMN
- The RIGHTMOST index changes FASTER than the leftmost when accessing elements in storage order
- Elements are laid out in memory ROW BY ROW (row-major order)

Memory size formula for a two-dimensional array:

```
bytes = size of 1st index × size of 2nd index × sizeof(base type)
```

```
/* Example: int array[10][5] with 4-byte integers: */
/* 10 × 5 × 4 = 200 bytes */
```

Array	Base Type	Calculation	Memory Used
int[10][5]	int (4 bytes)	$10 \times 5 \times 4$	200 bytes
char[3][4]	char (1 byte)	$3 \times 4 \times 1$	12 bytes
double[10][5]	double (8 bytes)	$10 \times 5 \times 8$	400 bytes

Passing Two-Dimensional Arrays to Functions

When a two-dimensional array is used as an argument to a function, only a pointer to the first element is actually passed. However, the parameter receiving a two-dimensional array **MUST** define at least the size of the RIGHTMOST dimension.

```
/* A function receiving a 2D integer array with right dimension = 10 */
void func1(int x[][10])
{
    /* ... */
}
```

- The rightmost dimension is required — the compiler needs to know the length of each row to correctly index the array
- Without knowing the row length, the compiler cannot determine where the next row begins in memory
- You can specify the left dimension if you like, but it is NOT necessary

Example — student grades database using a 2D array:

```
/* A simple student grades database */
#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>
#define CLASSES 3
#define GRADES 30
int grade[CLASSES][GRADES]; /* global 2D array */

void enter_grades(void);
int get_grade(int num);
void disp_grades(int g[][GRADES]); /* right dim REQUIRED */

/* Enter all student grades */
void enter_grades(void)
```

```

{
    int t, i;
    for(t=0; t<CLASSES; t++) {
        printf("Class # %d:\n", t+1);
        for(i=0; i<GRADES; ++i)
            grade[t][i] = get_grade(i);
    }
}

/* Read a single grade */
int get_grade(int num)
{
    char s[80];
    printf("Enter grade for student # %d:\n", num+1);
    gets(s);
    return(atoi(s));
}

/* Display all grades — receives 2D array pointer */
void disp_grades(int g[][GRADES])
{
    int t, i;
    for(t=0; t<CLASSES; ++t) {
        printf("Class # %d:\n", t+1);
        for(i=0; i<GRADES; ++i)
            printf("Student # %d is %d\n", i+1, g[t][i]);
    }
}

```

Arrays of Strings

It is not uncommon in programming to use an array of strings — for example, to verify user commands against an array of valid commands. To create an array of strings, use a two-dimensional character array.

Dimension	Controls
Left dimension (rows)	The number of strings in the array
Right dimension (columns)	The maximum length of each string (include space for null terminator)

```

char str_array[30][80]; /* 30 strings, each up to 79 chars + null */

/* Access an individual string — specify only the LEFT index */
gets(str_array[2]); /* reads into 3rd string (index 2) */

/* Equivalent to: gets(&str_array[2][0]); */
/* But first form is much more common in professional C code */

```

Example — a simple text editor using a string array:

```
/* A very simple text editor */
#include <stdio.h>
#define MAX 100
#define LEN 80
char text[MAX][LEN]; /* array of up to 100 strings, each 80 chars */
int main(void)
{
    register int t, i, j;
    printf("Enter an empty line to quit.\n");
    for(t=0; t<MAX; t++) {
        printf("%d: ", t);
        gets(text[t]);
        if(!*text[t]) break; /* quit on blank line */
    }
    /* redisplay each line one character at a time */
    for(i=0; i<t; i++) {
        for(j=0; text[i][j]; j++) putchar(text[i][j]);
        putchar('\n');
    }
    return 0;
}
/* This program inputs lines of text until a blank line is entered,
then redisplay each line one character at a time. */
```

MULTIDIMENSIONAL ARRAYS

C allows arrays of more than two dimensions. The general form of a multidimensional array declaration is:

```
type name[Size1][Size2][Size3]...[SizeN];
```

Memory Requirements

Arrays of more than three dimensions are not often used because of the amount of memory they require. The storage required increases EXPONENTIALLY with the number of dimensions.

Array Dimensions	Base Type	Memory Required
[10][6][9][4]	char (1 byte)	$10 \times 6 \times 9 \times 4 = 2,160$ bytes
[10][6][9][4]	int (2 bytes)	$10 \times 6 \times 9 \times 4 \times 2 = 4,320$ bytes
[10][6][9][4]	double (8 bytes)	$10 \times 6 \times 9 \times 4 \times 8 = 17,280$ bytes

Array Dimensions	Base Type	Memory Required
[10][6][9][4][10]	char (1 byte)	+ one dimension of size 10 → 172,800 bytes!

NOTE *In multidimensional arrays, it takes the computer time to compute each index. This means that accessing an element in a multidimensional array can be slower than accessing an element in a single-dimension array.*

Passing Multidimensional Arrays to Functions

When passing multidimensional arrays to functions, you must declare ALL but the LEFTMOST dimension in the receiving function's parameter list.

```
/* Array declared as: */
int m[4][3][6][5];

/* The receiving function must specify all but the leftmost: */
void func1(int d[][3][6][5])
{
    /* ... */
}
/* The first dimension can optionally be included if desired */
```

- ALL dimensions except the leftmost are required by the compiler
- The other dimensions are needed to allow the compiler to correctly index the array
- Including the first (leftmost) dimension is optional — it has no effect on compilation

Indexing Pointers

Pointers and arrays are closely related. An array name without an index is a pointer to the first element. Conversely, a pointer can be indexed as if it were declared to be an array.

```
char p[10];
/* The following statements are identical: */
p      /* array name = pointer to first element */
&p[0]  /* address of first element */
/* p == &p[0] evaluates to true */
```

A pointer can be used with both array indexing and pointer arithmetic — both produce the same result:

```
int *p, i[10];
p = i;
p[5] = 100; /* assign using array index */
*(p+5) = 100; /* assign using pointer arithmetic */
```

```
/* Both statements place 100 in the sixth element of i */
```

This concept also applies to two-dimensional arrays. For a 10×10 integer array a:

```
/* These two statements are equivalent: */
a
&a[0][0]

/* Accessing element [0][4]: */
a[0][4]      /* using array indexing */
*((int *)a + 4) /* using pointer arithmetic */

/* Accessing element [1][2]: */
a[1][2]      /* using array indexing */
*((int *)a + 12) /* using pointer arithmetic (1×10 + 2 = 12) */

/* General formula for any two-dimensional array: */
a[j][k] is equivalent to *((base_type *)a + (j × row_length) + k)
```

NOTE *The cast of the pointer to the array into a pointer of its base type is necessary for pointer arithmetic to operate properly. Pointers are sometimes used to access arrays because pointer arithmetic is often faster than array indexing.*

Using a pointer variable to access elements within a row of a 2D array:

```
int num[10][10];

/* Print contents of a specified row */
void pr_row(int j)
{
    int *p, t;
    p = (int *) &num[j][0]; /* get address of first element in row j */
    for(t=0; t<10; ++t) printf("%d ", *(p+t));
}

/* Generalized version — takes row, row dimension, and pointer to first element */
void pr_row(int j, int row_dimension, int *p)
{
    int t;
    for(t=0; t<row_dimension; ++t) printf("%d ", *(p+(j*row_dimension)+t));
}
```

A two-dimensional array can be reduced to a pointer to an array of one-dimensional arrays. Using a separate pointer variable is one easy way to use pointers to access elements within a row of a two-dimensional array.

ARRAY INITIALIZATION

C allows you to initialize arrays at the time of their declaration by placing an equal sign and a brace-enclosed, comma-separated list of values after the array name.

Single-Dimension Array Initialization

General form:

```
type array_name[size] = { value1, value2, ..., valueN };

/* Example: */
int i[5] = { 1, 2, 3, 4, 5 };
/* i[0]=1, i[1]=2, i[2]=3, i[3]=4, i[4]=5 */
```

String (Character Array) Initialization

Character arrays that hold strings allow a shorthand initialization using a string constant:

```
char array_name[size] = "string";

/* Example: initialize str to the phrase "I like C" */
char str[9] = "I like C";

/* This is identical to: */
char str[9] = {'I', ' ', 'l', 'i', 'k', 'e', ' ', 'C', '\0'};
```

- Because strings end with a null terminator, you must declare the array to be long enough to include the null
- str is 9 characters long even though "I like C" is only 8 characters — the 9th position holds '\0'
- When you use a string constant in initialization, the compiler automatically supplies the null terminator

Multidimensional Array Initialization

Multidimensional arrays are initialized the same as single-dimension ones. Initializers are assigned in row-major order (rightmost index changes fastest).

```
/* Basic form — initializes sqrs with numbers 1-10 and their squares */
int sqrs[10][2] = {
    1,  1,
    2,  4,
    3,  9,
    4, 16,
    5, 25,
```

```
6, 36,  
7, 49,  
8, 64,  
9, 81,  
10, 100  
};
```

Subaggregate Grouping

When initializing a multidimensional array, you may add braces around the initializers for each dimension. This is called subaggregate grouping. It makes the structure of the initialization clearer:

```
/* Same array using subaggregate grouping — braces around each row */  
int sqrs[10][2] = {  
    { 1, 1},  
    { 2, 4},  
    { 3, 9},  
    { 4, 16},  
    { 5, 25},  
    { 6, 36},  
    { 7, 49},  
    { 8, 64},  
    { 9, 81},  
    {10, 100}  
};
```

NOTE *When using subaggregate grouping, if you don't supply enough initializers for a given group, the remaining members will be set to zero automatically.*

Unsize Array Initializations

If the size of the array is not specified in an array initialization statement, the compiler automatically creates an array big enough to hold all the initializers present. This is called an unsize array.

Without unsize arrays — tedious and error-prone (must count characters manually):

```
char e1[12] = "Read error\n";  
char e2[13] = "Write error\n";  
char e3[18] = "Cannot open file\n";
```

With unsize arrays — compiler calculates dimensions automatically:

```
char e1[] = "Read error\n";  
char e2[] = "Write error\n";  
char e3[] = "Cannot open file\n";
```

```
/* Verify: */
printf("%s has length %d\n", e2, sizeof e2);
/* Output: Write error
   has length 13 (12 chars + null terminator) */
```

Advantages of unsized array initialization:

- Eliminates the tedious need to count characters manually
- Allows changing any message without worrying about incorrect array dimensions
- Less error-prone — the compiler always gets the size right

Unsized initializations are not restricted to one-dimensional arrays. For multidimensional arrays, you must specify all but the leftmost dimension:

```
/* Unsized multidimensional array — leftmost dimension omitted */
int sqrs[][2] = {
    { 1, 1},
    { 2, 4},
    { 3, 9},
    { 4, 16},
    { 5, 25},
    { 6, 36},
    { 7, 49},
    { 8, 64},
    { 9, 81},
    {10, 100}
};
/* The compiler calculates the left dimension automatically (= 10) */
/* Advantage: you can add or remove rows without changing the declaration */
```

NOTE *For multidimensional unsized arrays, you must still specify all dimensions except the leftmost. The other dimensions are needed to allow the compiler to correctly index the array.*

VARIABLE-LENGTH ARRAYS

In C89, array dimensions must be declared using constant expressions — the size of an array is fixed at compile time. However, C99 adds a powerful new feature: variable-length arrays (VLAs).

In C99, you can declare an array whose dimensions are specified by any valid expression, including those whose value is known only at run time. This is called a variable-length array.

```
/* Variable-length array — C99 only */
void f(int dim)
```

```

{
    char str[dim]; /* size of str is determined at run time by dim */
    /* ... */
}
/* Each call to f() can create str with a different length */
/* The size is determined by the value of dim passed to f() */

```

Feature	C89	C99
Array size at compile time	Required — size must be a constant expression	Optional — size can be any valid expression
Array size at run time	Not supported	Supported — variable-length arrays (VLAs)
Scope restriction for VLAs	N/A	Only local arrays (block scope or prototype scope) can be variable-length; global or static arrays cannot be VLAs
Support in C++	N/A	VLAs are NOT supported by C++

- The size of a VLA is determined by the value passed at run time — each call can result in a different length
- One major reason for VLAs in C99 is to support numeric processing
- VLAs have widespread applicability but are not supported by C89 or C++
- Only local arrays (block scope or prototype scope) can be of variable length

QUICK REFERENCE SUMMARY — CHAPTER 4

Array Declarations

Declaration	Meaning
type name[size];	Single-dimension array with 'size' elements
type name[rows][cols];	Two-dimensional array — rows × cols elements
type name[S1][S2][S3];	Three-dimensional array
type name[rows][];	Invalid — leftmost can be omitted only in initialization or function params
type name[];	Unsize array — valid only in initialization context

Key Array Rules

Rule	Detail
Index starts at 0	First element is always [0], last is [size-1]
No bounds checking	C does not check array boundaries — programmer's responsibility
Memory: 1D	total bytes = sizeof(base type) × length
Memory: 2D	bytes = rows × cols × sizeof(base type)
Pointer to array	Array name without index = pointer to first element
Pass to function	Array name without index passed; received as pointer
1D parameter	int *x or int x[] or int x[n] — all equivalent
2D parameter	Must specify rightmost dimension: int x[][cols]
Multi-D parameter	Must specify all but leftmost dimension
String	Null-terminated char array; declare 1 char extra for '\0'
String init shorthand	char s[9] = "I like C"; — compiler adds null
Unsize init	char s[] = "hello"; — compiler computes size
Subaggregate grouping	Use { } around each row: int a[2][2] = {{1,2},{3,4}};
VLAs (C99 only)	Local arrays only; size is a run-time expression

String Functions Quick Reference (Header: <string.h>)

Function	Action
strcpy(s1, s2)	Copy s2 into s1
strcat(s1, s2)	Append s2 to end of s1
strlen(s1)	Return length of s1 (excluding null)
strcmp(s1, s2)	Compare s1 and s2 (0=equal, <0=s1 less, >0=s1 greater)
strchr(s1, ch)	Find first occurrence of ch in s1 — returns pointer or NULL
strstr(s1, s2)	Find first occurrence of s2 in s1 — returns pointer or NULL

— End of Chapter 4 Notes —